BTK AKADEMİ

Her an,
her yerde
öğren !

Yapay Zeka ve Üretken Modeller Atölyesi - 5

Dr.Öğr.Üyesi Furkan Göz

Görüntü İşleme ve Derin Öğrenme İlişkisi

- Evrişimsel sinir ağları ham piksel matrisleri üzerinden özellik çıkarımı yapar.
- Görüntü işleme algoritmaları ham veriyi modelin daha kolay işleyebileceği formata dönüştürür.
- Piksel matrislerindeki gürültüleri temizlemek ve hedef nesneleri belirginleştirmek temel amaçtır.

Kullanım Amaçları

- Veri setinin yapısal kalitesini artırır.
- Ağın optimizasyon sürecini hızlandırır.
- YOLO türevi nesne tespiti modellerine nitelikli girdi sağlar.

Temel Görüntü İşleme Teknikleri (1/2)

- **Grayscale:** Görüntü matrisini üç kanallı RGB yapıdan tek kanallı yapıya indirger. Renk bilgisinin önemsiz olduğu yapısal analizlerde tercih edilir.
- **Gaussian ve Median Blur:** Görüntü matrisi üzerindeki istenmeyen hatalı pikselleri gidermek amacıyla uygulanan filtrelerdir.
- **Thresholding (Eşikleme):** Piksel değerlerini belirlenen eşik değerine göre siyah ve beyaz renklere ayırır. Otsu algoritması otomatik eşik hesaplama yöntemlerine örnektir.

Temel Görüntü İşleme Teknikleri (2/2)

- **Canny Kenar Tespiti:** Görüntü içindeki nesnelerin ayırt edici yapısal sınırlarını matematiksel olarak belirler.
- **Sobel ve Laplacian:** Pikseller arasındaki yatay ve dikey renk geçiş şiddetlerini hesaplar.
- **Kontur Tespiti:** Tespit edilen kenar piksellerini birbirine bağlayarak kapalı şekil formatlarına dönüştürür.
- **Histogram Eşitleme:** Matris üzerindeki ışık dağılımını dengeler ve düşük ışıklı görsellerin genel kontrast oranını artırır.

Morfolojik ve Geometrik İşlemler

- **Erozyon ve Genişletme:** Nesne piksel sınırlarını küçültme ve büyütme işlemleridir. Gürültü temizleme ve yapısal boşluk doldurma görevlerinde uygulanır.
- **Açma ve Kapama:** Piksellerdeki ince çıkıntıları yumuşatma ve nesne içindeki boşlukları kapatma fonksiyonlarıdır.
- **Boyutlandırma ve Kırpma:** Görüntü matrisini sinir ağı giriş katmanı boyutlarına sabitleyen yapısal adımlardır.
- **Geometrik Dönüşümler:** Görüntü eksenlerini eğme ve perspektif düzeltme işlemlerini içerir.

Görüntü İşleme Kullanım Alanları

- **Plaka Tanıma:** Siyah beyaz dönüşüm, eşikleme ve kenar tespiti algoritmaları karakter sınırlarını çıkarır.
- **Belge Tarama (OCR):** Perspektif düzeltme ve ikili eşikleme işlemleri metin okuma başarı oranını yükseltir.
- **Nesne Takibi:** Görüntü matrisi HSV renk uzayına dönüştürülür ve renk aralığı filtresi uygulanır.
- **Derin Öğrenme Ön İşleme:** Boyutlandırma ve normalizasyon adımları CNN ağlarından önce veriyi eğitime hazır hale getirir.

Transfer Learning Nedir?

Eğitilmiş Bilginin Yeniden Kullanımı

- Derin öğrenme ağlarını rastgele ağırlıklarla sıfırdan eğitmek yerine önceden optimize edilmiş model parametrelerini yeni problemlerde kullanma mimarisidir.

Transfer Learning ve Görüntü Sınıflandırma

- Bilgisayarlı görü görevlerinde temel standart model geliştirme süreci olarak kabul edilir.
- Literatürde ResNet, MobileNet, VGG ve EfficientNet önceden eğitilmiş temel ağ mimarileridir.
- Modeller başlangıç ağırlıklarını ImageNet veri seti üzerindeki eğitimlerden elde eder.
- Aktarılan modelin son katmanları hedef veri seti ile yeniden eğitilerek probleme uyarlanır.

```
from tensorflow.keras.applications import ResNet50
model = ResNet50(weights='imagenet', include_top=False)
```

ImageNet Veri Kümesi

- ImageNet 14 milyon etiketli görsel veri içeren geniş ölçekli bir akademik veri kümesidir.
- Toplam sınıf kapasitesi 21 bin üzerinde kategori barındırır.
- İlgili veri setinin 1000 sınıflı özel alt kümesi standart ağ eğitimlerinde temel referans kabul edilir.
- VGG ve ResNet mimarileri temel görsel özellikleri bu veri seti üzerinden öğrenir.
- Aktarılan parametreler sayesinde yeni görüntü görevlerinde sınıflandırma bilgisi doğrudan transfer edilir.

Önceden Eğitilmiş Modeller (Pre-trained Models)

- Önceden eğitilmiş modeller genel kapsamlı yapılara sahiptir ve çeşitli donanım görevleri için uyarlanabilir.
- uygulamalarda doğrudan çıkarım aracı veya ince ayar (fine tuning) mimarisi olarak değerlendirilir.
- 1000 ayrı nesne sınıfını inceleyen ağ kenar ve doku özelliklerini matematiksel olarak öğrenir.
- ImageNet üzerinde genel nesneleri öğrenen bir ağ özelleşmiş medikal görüntüleme problemlerine aktarılabilir.
- Önceden öğrenilmiş yapı düşük veri miktarı ile istikrarlı metrik sonuçları üretilmesini sağlar.

Transfer Learning Temel Mantığı

- Gelişmiş derin öğrenme ağları milyonlarca optimizasyon parametresi barındırır.
- Kapsamlı modellerin eğitimi geniş veri setleri ve yüksek hesaplama donanımı gerektirir.
- Transfer öğrenimi sistem sınırlandırmalarını aşmak için formüle edilmiştir.

İşlem Akışı

- Temel görsel özellikleri barındıran CNN mimarisi çalışma dizinine aktarılır.
- Ağın özellik çıkaran erken katmanları dondurulur ve gradyan güncellemeleri kapatılır.
- Modelin son katmanları yeni etiket formatına göre yapılandırılarak eğitime açılır.

Neden Transfer Learning Kullanılır?

- Ağ parametrelerini rastgele başlangıç değerleriyle eğitmek yüksek sistem gereksinimleri oluşturur.
- Orijinal eğitim süreçleri milyonlarca etiketli görsele ve yoğun grafik işlemci sürelerine ihtiyaç duyar.
- Akademik ve senaryolarda bu veri büyüklüğü standart olarak sağlanamaz.
- Optimize edilmiş ağırlıkların aktarımı kısıtlı veri setlerinde doğruluk oranını artırır.
- Model öğrenme süresini optimize eder ve aşırı öğrenme ihtimalini sınırlar.

Transfer Learning Uygulama Aşamaları

1. **Hazır Modelin Yüklenmesi:** ImageNet üzerinde eğitilmiş MobileNet veya ResNet gibi ağırlıklar sisteme tanımlanır.
2. **Çıkış Katmanlarının Değiştirilmesi:** Orijinal sınıflandırma katmanı silinir ve hedef sınıf sayısına uygun yeni bir yoğun katman (Dense) eklenir.
3. **Eğitim Stratejisinin Belirlenmesi:**
   - **Özellik Çıkarımı (Feature Extraction):** Aktarılan tüm katmanlar dondurulur ve sadece yeni eklenen katman eğitilir.
   - **İnce Ayar (Fine Tuning):** Ağın temel katmanları dondurulurken son bloklarda yer alan katmanlar yeni veri setiyle eğitime açılır.

Keras Applications ve Hazır Modeller

- Keras kütüphanesi literatürde standart olarak kabul edilen ağ mimarilerini projeye dahil etme arayüzü sunar.

Temel Mimariler

- **MobileNetV2:** Mobil cihazlar ve düşük işlemci kapasiteli donanımlar için optimize edilmiş ağ yapısıdır.
- **ResNet50:** Kalıntı (residual) blokları içeren mimarisi ile yüksek gradyan akışı sağlar.
- **EfficientNet:** Ağın derinlik, genişlik ve çözünürlük parametrelerini orantılı şekilde büyütür.

```
from tensorflow.keras.applications import MobileNetV2
base_model = MobileNetV2(weights='imagenet', include_top=False)
```

Transfer Learning Stratejileri

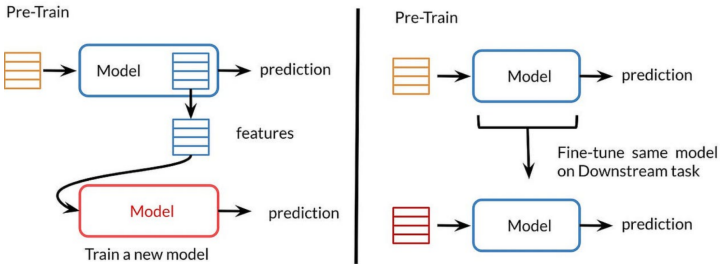
1. Özellik Çıkarımı (Feature Extraction)

- Önceden eğitilen katmanların tamamı dondurulur ve donanım ağırlık optimizasyonlarını durdurur.
- Modelin sonuna eklenen yeni sınıflandırma bloğu tek başına eğitilir.
- Veri seti küçük olduğunda ezberlemeyi önlemek için tercih edilen yaklaşımdır.

2. İnce Ayar (Fine Tuning)

- Önceden eğitilen katmanların belirli bir bölümü eğitime açılarak ağırlık matrisleri güncellenir.
- Model yeni veri setindeki piksel yapılarına hassas uyum sağlar.
- Sistem donanımında yeterli miktarda eğitim verisi bulunduğunda uygulanır.

Transfer Learning Mimarisi



Özellik Çıkarımı ve İnce Ayar Karşılaştırması

- Düşük veri miktarında özellik çıkarımı yöntemi uygulanır.
- Yüksek veri miktarında ince ayar yöntemi ağ performansını artırır.
- İnce ayar sürecinde ağırlık güncellemelerini korumak için düşük öğrenme oranı seçilir.

Uygulama Kriterleri

- Sınırlı örnek barındıran veri setlerinde sadece son katmanları eğitmek aşırı öğrenmeyi engeller.
- Kapsamlı örnek barındıran projelerde temel katman blokları da eğitime açılabilir.

Transfer Learning Uygulama Süreci

1. Önceden eğitilmiş model yapılandırması yüklenir.
2. Modele ait son sınıflandırma katmanları çıkartılır.
3. Yeni tam bağlı katman ve çıkış katmanı eklenir.
4. Temel katman ağırlıkları dondurulur ve ağ derlenerek eğitilir.
5. Gerekli durumlarda temel katmanlar eğitime açılarak ince ayar işlemi gerçekleştirilir.

MobileNetV2 Taban Modelinin Yüklenmesi

- ImageNet üzerinde önceden eğitilmiş temel model donanıma aktarılır.
- Orijinal sınıflandırma katmanı silinerek sadece özellik çıkaran matris blokları alınır.

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import
    preprocess_input

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
```

Not: MobileNetV2 varsayılan giriş boyutu esnektir ancak pratikte 224x224 piksel formları tercih edilir.

Önceden Eğitilmiş Katmanların Dondurulması

- İlk eğitim aşamasında temel modelin ağırlıkları dondurulur.
- İlgili katmanlar güncellenmez ve donanım hesaplama yükü azaltılır.
- Sınırlı veri setlerinde modelin ezberleme (overfitting) riski düşürülür.

```
base_model.trainable = False
print("Dondurulan katman sayisi:", len(base_model.layers))
```

Eğitimden önce model özeti fonksiyonu ile katman durumları kontrol edilebilir.

Sınıflandırma Katmanlarının Eklenmesi

- Taban model çıktıları üzerine yeni tam bağlı (Dense) katmanlar eklenir.
- Küresel havuzlama katmanı uzamsal matrisleri boyutsuz vektörlere indirger.
- Seyreltme (dropout) işlemi ile aşırı öğrenme problemi engellenir.

```
from tensorflow.keras import layers, models

num_classes = 5
inputs = layers.Input(shape=(224, 224, 3))
x = preprocess_input(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)

model = models.Model(inputs, outputs)
```

Özellik Çıkarımı Eğitim Aşaması

- Taban katmanlar dondurulur ve sadece yeni eklenen tam bağlı katman optimize edilir.
- Taban model güncellenmediği için varsayılan optimizasyon algoritmaları doğrudan uygulanabilir.

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'])

history_fe = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5
)
```

İnce Ayar (Fine Tuning) İşlemi

- Özellik çıkarımı aşamasından sonra modelin belirli katmanları eğitime açılır.
- Yüksek miktarda katmanı aynı anda güncellemek kısıtlı veri setlerinde optimizasyon hataları yaratır.

```
base_model.trainable = True
fine_tune_at = len(base_model.layers) - 30

for i, layer in enumerate(base_model.layers):
    layer.trainable = (i >= fine_tune_at)
```

İnce Ayar Eğitimi ve Öğrenme Oranı

- Önceden öğrenilmiş ağırlıkların bozulmasını önlemek amacıyla düşük bir öğrenme oranı tanımlanır.
- Özellik çıkarımı aşamasının ardından ağ optimizasyonu kaldığı noktadan devam eder.

```
from tensorflow.keras.optimizers import Adam

model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy'])

history_ft = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10)
```

Erken Durdurma ve Model Kaydetme

- Doğrulama kayıp değeri iyileşme göstermediğinde eğitim süreci otomatik olarak sonlandırılır.
- Eğitim dönemi boyunca elde edilen en isabetli ağırlıklar kayıt dizinine aktarılır.

```
from tensorflow.keras.callbacks import EarlyStopping,
    ModelCheckpoint
callbacks = [EarlyStopping(monitor='val_loss', patience=3,
    restore_best_weights=True),  ModelCheckpoint('best_model.
    keras', monitor='val_loss', save_best_only=True)]
history_ft = model.fit(
     train_ds,
     validation_data=val_ds,
     epochs=20, callbacks=callbacks)
```

Eğitim Sonuçlarının Görselleştirilmesi

```
import matplotlib.pyplot as plt

def plot_history(h, label):
    plt.plot(h.history['accuracy'], label=f'{label} Egitim')
    plt.plot(h.history['val_accuracy'], label=f'{label}
        Dogrulama')

plt.figure()
plot_history(history_fe, 'Ozellik Cikarimi')
plot_history(history_ft, 'Ince Ayar')
plt.xlabel('Epoch')
plt.ylabel('Dogruluk')
plt.legend()
plt.show()
```

Transfer Learning Uygulama Yöntemleri

- Aktarım öğrenimi ağın mimarisine ve veri setinin yapısına göre farklı stratejilerle uygulanır.

Temel Yaklaşımlar

1. Sabit Özellik Çıkarıcı
2. İnce Ayar
3. Tam Model Eğitimi
4. Kısmi Katman Dondurma
5. Alan Uyarlama

Uygulama türü donanım kapasitesine ve proje hedeflerine göre belirlenir.

1. Sabit Özellik Çıkarıcı Yöntemi

- Önceden eğitilmiş bir sinir ağı modeli çalışma dizinine aktarılır.
- Mimari içindeki tüm temel katmanlar dondurularak ağırlık güncellemeleri kapatılır.
- Modelin sonuna eklenen çıkış katmanları hedef etiketlere göre eğitilir.

Kullanım Kriterleri

- Düşük örnek sayısına sahip veri setlerinde tercih edilir.
- Hızlı sonuç üretir ve işlem maliyetini düşürür.

2. İnce Ayar (Fine Tuning) Yöntemi

- Modelin son katmanlarına ek olarak belirli temel bloklar da eğitime açılır.
- Ağ yapısı yeni veri setinin matematiksel özelliklerine yüksek oranda uyum gösterir.

Uygulama Adımları

- Başlangıç aşamasında tüm temel katmanlar dondurulur ve uç katmanlar eğitilir.
- İkinci aşamada belirlenen katmanların dondurulma işlemi iptal edilir.
- Ağırlıkların bozulmasını önlemek amacıyla düşük bir öğrenme oranı ile eğitim sürdürülür.

3. Tam Model Eğitimi

- Önceden eğitilmiş model ağırlıkları başlangıç referansı olarak sisteme tanımlanır.
- Mimarideki tüm katmanlar dondurulmadan doğrudan eğitime açılır.

Kullanım Kriterleri

- Yüksek hacimli veri setleri kullanıldığında tercih edilir.
- Eğitim donanımı yeterli işlem kapasitesine sahip olduğunda uygulanır.
- Ağ veri setine uyum sağlar ancak optimizasyon süresi uzar.

4. Kısmi Katman Dondurma Yöntemi

- Öğrenme sürecinde ağın belirli katmanları dondurulur ve hedeflenen katmanlar serbest bırakılarak eğitime alınır.
- Önceden öğrenilmiş genel özellikleri korumak ve hedef veri setine özel katmanları güncellemek temel amaçtır.

Uygulama Adımları

- Ağ parametreleri yüklenerek tüm katmanlar dondurulur.
- Belirlenen temel bloklar eğitime açılır.
- Düşük öğrenme oranı ile optimizasyon işlemi başlatılır.

Sağlanan Avantajlar

- Optimizasyon süresini kısaltır.
- Aşırı öğrenme riskini azaltır.
- Aktarılan ağırlık bilgilerini korur.

5. Alan Uyarlama (Domain Adaptation)

- Kaynak veri seti ve hedef veri seti benzer görevleri barındırıp farklı veri dağılımlarına sahip olduğunda uygulanır.
- Gündüz çekilen görsellerle eğitilen modelin gece görsellerine uyarlanması alan uyarlamasına örnektir.
- Ağın genelleme yeteneğini artırmak hedeflenir.

Temel Yöntemler

- Normalizasyon ve standardizasyon teknikleri
- Stil aktarımı işlemleri
- Çekişmeli öğrenme yaklaşımları

Model Karşılaştırma Parametreleri

- **Model:** İlgili derin öğrenme mimarisinin ismini belirtir.
- **Boyut (MB):** Ağın depolama biriminde kapladığı alanı gösterir.
- **Top-1 Doğruluk:** Ağın en yüksek olasılıklı tahmininin doğru çıkma oranını hesaplar.
- **Top-5 Doğruluk:** Doğru sınıf etiketinin en yüksek olasılıklı beş tahmin arasında bulunma oranını ölçer.
- **Parametreler:** Ağ içinde optimize edilen ağırlık ve sapma değerlerinin toplam sayısını verir.
- **Derinlik:** Katman sayısını belirtir.
- **Çıkarım Süresi (CPU/GPU):** Sistemin tek bir görseli sınıflandırmak amacıyla harcadığı milisaniye cinsinden süreyi yansıtır.

Model Performansları 1

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7

Model Performansları 2

EfficientNetB0	29	77.1%	93.3%	5.3M	132	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	186	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	186	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	210	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	258	308.3	15.1
EfficientNetB5	118	83.6%	96.7%	30.6M	312	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	360	958.1	40.4
EfficientNetB7	256	84.3%	97.0%	66.7M	438	1578.9	61.6
EfficientNetV2B0	29	78.7%	94.3%	7.2M	-	-	-
EfficientNetV2B1	34	79.8%	95.0%	8.2M	-	-	-
EfficientNetV2B2	42	80.5%	95.1%	10.2M	-	-	-
EfficientNetV2B3	59	82.0%	95.8%	14.5M	-	-	-
EfficientNetV2S	88	83.9%	96.7%	21.6M	-	-	-
EfficientNetV2M	220	85.3%	97.4%	54.4M	-	-	-
EfficientNetV2L	479	85.7%	97.5%	119.0M	-	-	-
ConvNeXtTiny	109.42	81.3%	-	28.6M	-	-	-
ConvNeXtSmall	192.29	82.3%	-	50.2M	-	-	-
ConvNeXtBase	338.58	85.3%	-	88.5M	-	-	-
ConvNeXtLarge	755.07	86.3%	-	197.7M	-	-	-
ConvNeXtXLarge	1310	86.7%	-	350.1M	-	-	-

Model Seçim Kriterleri

Önceden eğitilmiş model seçimi doğruluk, donanım kapasitesi ve işlem hızı sınırlamalarına göre gerçekleştirilir.

Kullanım Alanına Göre Önerilen Mimariler

- **Mobil ve Gömülü Sistemler:** MobileNet, MobileNetV2 ve EfficientNetB0
- **Yüksek Doğruluk İhtiyacı:** EfficientNetB7, ResNet152V2 ve ConvNeXtLarge
- **Hız Odaklı Çıkarım İşlemleri:** ResNet50V2 ve EfficientNetB1
- **Düşük Parametre Maliyeti:** DenseNet121 ve MobileNet

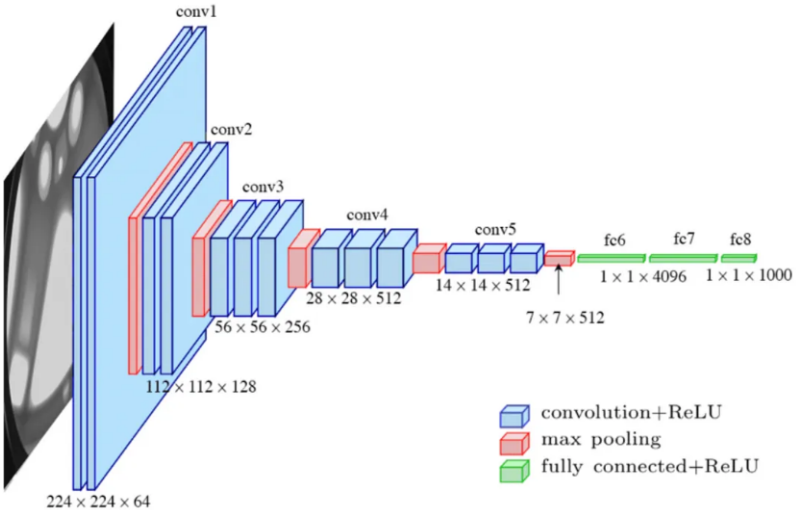
Model Optimizasyonunda Değerlendirme Adımları

- **Doğruluk Hedefi:** Sistemin asgari hata toleransı analiz edilir.
- **Donanım Kapasitesi:** Derin ağlar grafik işlemci gerektirir. Sınırlı donanımlarda kompakt ağlar tercih edilir.
- **İşlem Amacı:** Sistemi sıfırdan eğitmek yüksek kapasite gerektirir. Sadece çıkarım yapılacaksa daha az parametreli modeller sisteme dahil edilir.
- **Çalışma Ortamı:** Web tarayıcı, mobil sistem ve gömülü aygıt platformları kaynak tüketimini sınırlar.

VGG-16 Ağ Mimarisi

- 2014 yılında literatüre kazandırılmış standart evrişimsel sinir ağı modelidir.
- 13 evrişim katmanı ve 3 tam bağlı katman olmak üzere 16 öğrenilebilir katmandan oluşur.
- Ardışık 3x3 boyutlu küçük filtrelerin kullanılması ağın derinleşmesini sağlar.
- Yaklaşık 138 milyon öğrenilebilir parametre içerir.
- ImageNet veri setinde yüksek doğruluk değerleri üretir.
- Daha az donanım parametresi ile karmaşık görsel özellikleri tespit eder.
- Çıkarılan görsel matris temsilleri aktarım öğrenimi süreçlerinde yaygın olarak temel alınır.

VGG-16 Model Yapısı



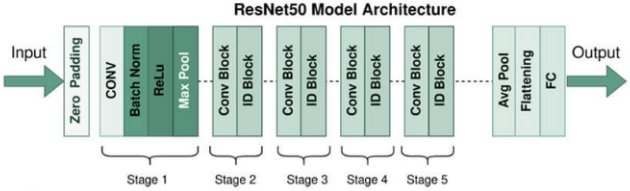
ResNet50 Ağ Mimarisi

- Ağ yapılarında katman derinliği arttıkça ortaya çıkan performans düşüşünü gidermek amacıyla tasarlanmıştır.
- Toplam 49 evrişim katmanı ve 1 tam bağlı çıkış katmanı içerir.
- Kalıntı bağlantıları kullanılarak ağ katmanları arasındaki gradyan kaybı en aza indirilir.
- Kalıntı blokları giriş verisinin bir kopyasını ileri katmanlara aktararak matematiksel optimizasyonu kolaylaştırır.
- ImageNet veri setindeki nesne tanımlama görevlerinde standart referans model olarak kabul görür.

ResNet50 Katman Düzeni

- Model mimarisi beş ana blok aşamasından oluşur.
- İlk aşama standart evrişim, normalizasyon ve havuzlama katmanlarını barındırır.
- Sonraki aşamalar yapısal boyut değiştiren evrişim blokları ile boyutu koruyan kalıntı bloklarını içerir.
- Son katmanlarda bölgesel havuzlama, düzleştirme ve sınıflandırma işlemleri yer alır.
- Sisteme entegre edilen özel köprü bağlantıları ağ derinliğinin 50 katmana çıkmasına rağmen gradyan akışını korur.

ResNet50 Model Yapısı



EfficientNet Mimarisi

- Minimum sistem parametresi kullanarak en yüksek doğruluğu elde etmeyi hedefleyen ağ mimarisidir.
- Model ölçeklendirme işlemlerini orantılı olarak yapılandırır.
- **Derinlik:** Katman sayısını artırarak ağı büyütür.
- **Genişlik:** Katmanlardaki filtre sayısını artırır.
- **Çözünürlük:** Matris giriş alanını genişletir.
- Belirtilen üç yapısal faktör matematiksel olarak birlikte optimize edilerek donanım verimliliği sağlanır.

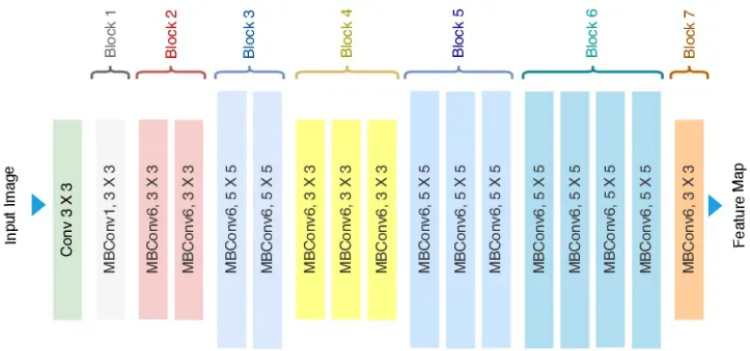
EfficientNet Ölçeklendirme Serisi

- EfficientNet algoritma ailesi B0 sürümünden B7 sürümüne kadar sekiz farklı ağ modeli sunar.
- **B0 Modeli:** Mobil cihazlara uygun, düşük donanım tüketen ve hızlı işlem yapan temel sürümdür.
- **B7 Modeli:** En yüksek doğruluk metriklerini sunan geniş parametreli sürümdür.

Giriş Matrisi ve Parametre Farklılıkları

- **B0 Sürümü:** 224x224 piksel giriş ve 5.3 milyon ağırlık katsayısı içerir.
- **B4 Sürümü:** 380x380 piksel giriş ve 19 milyon ağırlık katsayısı içerir.
- **B7 Sürümü:** 600x600 piksel giriş ve 66 milyon ağırlık katsayısı içerir.

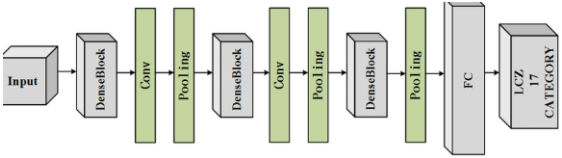
EfficientNet Ölçeklendirme Yapısı

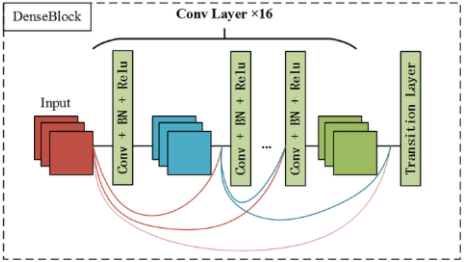


DenseNet Ağ Mimarisi

- DenseNet (Densely Connected Convolutional Networks) 2017 yılında literatüre sunulmuştur.
- Her katman kendinden önceki tüm katmanların çıktı matrislerini giriş verisi olarak kabul eder.
- Bilgi kaybını önlemek, gradyan akışını güçlendirmek ve parametre verimliliği sağlamak temel hedeflerdir.
- İleri seviye katmanlar erken katmanlardan gelen ham piksellere doğrudan erişim sağlar.
- Daha düşük parametre hacmi ile yüksek doğruluk metrikleri üretir.

DenseNet Model Yapısı





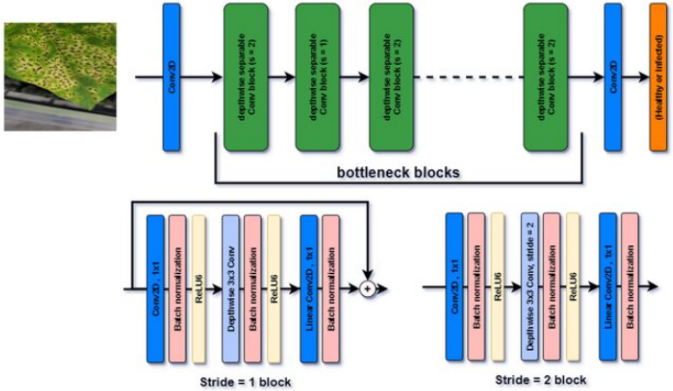
MobileNetV2 Mimarisi

- MobileNetV2 mobil donanımlar ve gömülü sistemler için optimize edilmiş bir ağ mimarisidir.
- Düşük hesaplama kapasitesine sahip cihazlarda görüntü işleme görevleri için tasarlanmıştır.
- Tersine çevrilmiş kalıntı blokları (Inverted Residual Block) kullanarak veri işleme hızını artırır.
- Sınırlı parametre sayısına rağmen yüksek doğruluk oranları sağlar.
- Mobil işletim sistemlerindeki anlık görüntü analizlerinde standart olarak tercih edilir.

MobileNetV2 Katman Düzeni

- Modelin temel işlem birimi ayrılabilir derinlik evrişimi (Depthwise Separable Convolution) tekniğidir.
- Her renk kanalı bağımsız filtrelenir ve sonrasında bire bir boyutlu evrişim katmanları ile birleştirilir.
- Tersine çevrilmiş kalıntı yapısı giriş matrisini önce genişletir ve ardından daraltır.
- İşlem adımları sırasıyla genişletme, derinlik evrişimi ve sıkıştırma aşamalarından oluşur.
- Sistem donanım üzerindeki matematiksel işlem maliyetini asgari seviyeye indirir.

MobileNetV2 Model Yapısı



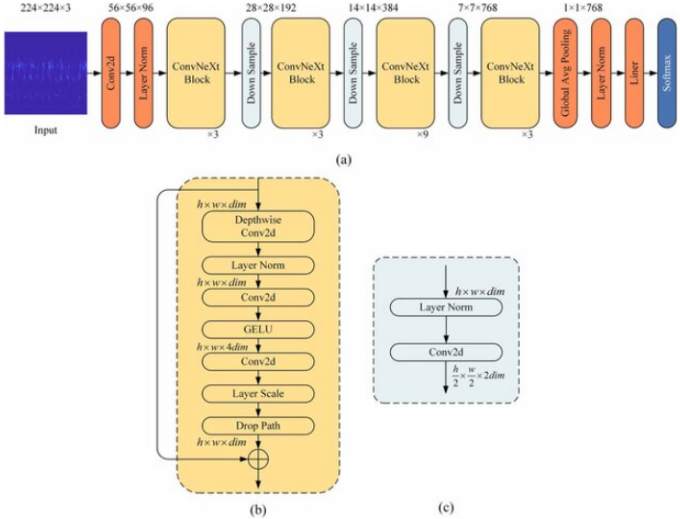
ConvNeXt Mimarisi

- ConvNeXt ağı 2022 yılında Meta AI araştırmacıları tarafından literatüre kazandırılmıştır.
- Temel hedef evrişimsel ağları Vision Transformer modellerinin başarı seviyesine ulaştırmaktır.
- Klasik ResNet altyapısını baz alır ve modern matematiksel güncellemeler içerir.
- Katman normalizasyonu, GELU aktivasyon fonksiyonu ve geniş boyutlu matris filtreleri kullanır.
- Görüntü işleme görevlerinde yüksek hız ve verimlilik dengesi sunar.

ConvNeXt Katman Düzeni

- Ağ tasarımı ResNet mimarisinde bulunan dört aşamalı blok yapısını korur.
- Giriş matrisi Vision Transformer ağlarına benzer şekilde bağımsız kare parçalara ayrılır.
- Geleneksel toplu normalizasyon yerine veri dizilimini dengeleyen katman normalizasyonu uygulanır.
- Aktivasyon sürecinde standart ReLU fonksiyonu yerine GELU algoritması tercih edilir.
- Evrişim ağlarının yerel algı avantajını Transformer ağlarının kapasitesiyle birleştirir.

ConvNeXt Model Yapısı



Güncel Yaklaşım: Vision Transformers (ViT)

- Görüntü işleme literatüründeki en güncel standartlardan biri Vision Transformer (ViT) mimarisidir.
- Görüntü matrisini klasik CNN ağları gibi pikseller üzerinden değil bağımsız kare parçalar (patch) üzerinden inceler.
- Doğal dil işleme alanındaki öz dikkat (Self-Attention) mekanizmasını görüntü işleme görevlerine uyarlar.
- Matris üzerindeki birbirine uzak piksellerin anlamsal ilişkisini ve bağlamını doğrudan hesaplar.
- Geniş veri setleriyle eğitildiğinde klasik evrişimsel ağlardan daha yüksek doğruluk metrikleri üretir.

Swin Transformer ve Modern Gelişmeler

- Vision Transformer mimarisinin yüksek hesaplama maliyetini düşürmek amacıyla Swin Transformer yapısı geliştirilmiştir.
- Görüntü parçalarını hiyerarşik pencereler halinde birleştirerek yerel ve küresel özellikleri aynı anda öğrenir.
- Nesne tespiti ve görüntü bölütleme (segmentation) görevlerinde güncel endüstri standardı haline gelmiştir.
- Modern görüntü işleme projeleri yüksek başarı için CNN ve Transformer mimarilerinin melez versiyonlarını kullanır.

Derin Öğrenme İçin Gerekli Kütüphaneler

- TensorFlow ve Keras kütüphaneleri çalışma ortamına dahil edilir.
- MobileNetV2 mimarisi önceden eğitilmiş ağırlıklarla projeye aktarılır.

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense,
    GlobalAveragePooling2D
from tensorflow.keras.models import Model
```

Veri Kümesinin İndirilmesi

```
import tensorflow as tf
import pathlib

dataset_url = 'https://storage.googleapis.com/download.
    tensorflow.org/example_images/flower_photos.tgz'
data_dir = tf.keras.utils.get_file('flower_photos', origin=
    dataset_url, untar=True)
data_dir = pathlib.Path(data_dir)

if (data_dir / 'flower_photos').exists():
     data_dir = data_dir / 'flower_photos'

print("Siniflar:", [item.name for item in data_dir.glob("*") if
    item.is_dir()])
```

Veri Kümesi Hazırlığı

```
batch_size = 32
img_size = (160, 160)

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training", seed=123,
    image_size=img_size,
    batch_size=batch_size)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation", seed=123,
    image_size=img_size,
    batch_size=batch_size)
```

Örnek Görsellerin Çizdirilmesi

```
class_names = sorted([item.name for item in data_dir.glob("*")
    if item.is_dir()])
plt.figure(figsize=(15, 5))

for i, class_name in enumerate(class_names):
    image_path = list((data_dir / class_name).glob("*.jpg"))[0]
    img = plt.imread(image_path)
    plt.subplot(1, len(class_names), i + 1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Veri Setinden Görsel Örnekleri

daisy

dandelion

roses

sunflowers

tulips

MobileNetV2 Tabanlı Model Kurulumu

- Temel model ImageNet ağırlıkları ile sisteme yüklenir ve son sınıflandırma katmanı çıkarılır.
- Taban ağırlıklar dondurularak yeni eklenen katmanlar hedef veri setine uyarlanır.

```
base_model = MobileNetV2(input_shape=(160, 160, 3),
                         include_top=False,
                         weights='imagenet')
base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(5, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=predictions)
```

Aktarım Modeli Mimari Analizi

- **include_top=False** parametresi orijinal sınıflandırma vektörünü iptal eder.
- **trainable=False** parametresi temel modelin ağırlık optimizasyonunu durdurur.
- **GlobalAveragePooling2D** katmanı çok boyutlu özellikleri tek boyutlu vektörlere indirger.
- **Dense(128)** parametresi özellik haritası üzerinde yeni bağlantılar kurar.
- **Dense(5)** katmanı veri setindeki 5 farklı çiçek sınıfı için hesaplanmış sınıf olasılıklarını üretir.

Modelin Derlenmesi ve Eğitimi

- Eğitim aşamasında sadece yeni eklenen son katmanlar optimize edilir.
- Kayıp fonksiyonu ve optimizasyon algoritması tanımlanarak eğitim süreci başlatılır.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history_fe = model.fit(train_ds,
                       validation_data=val_ds,
                       epochs=5)
```

İnce Ayar (Fine Tuning) İşlemi

- Ağın son birkaç katmanı eğitime açılarak model performansı hedef veriye göre optimize edilir.

```
base_model.trainable = True

fine_tune_at = 100
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history_ft = model.fit(train_ds,
                       validation_data=val_ds,
                       epochs=5)
```

İnce Ayar Parametre Analizi

- **base_model.trainable = True** komutu ile MobileNetV2 mimarisinin tüm katmanları eğitime açılır.
- **fine_tune_at = 100** komutu ile ilk 100 katman dondurulur ve kalan ileri katmanlar eğitilir.
- Öğrenilmiş temel özellikler korunur ve ağ yeni veri setine matematiksel olarak uyum sağlar.
- **Adam(1e-5)** parametresi ağırlıkların yavaş güncellenmesi için düşük bir öğrenme oranı tanımlar.
- **epochs=5** parametresi ile optimizasyon işlemi beş döngü boyunca sürdürülür.

İnce Ayar İşleminin Zamanlaması

- Önceden eğitilmiş ağlar milyonlarca görüntü verisi ile genel özellikler üzerinden optimize edilmiştir.
- Ağırlıklar doğrudan değiştirildiğinde modelin genel görsel algı kapasitesi düşer.
- Aktarım öğrenimi süreçleri veri kayıplarını önlemek için iki aşamalı olarak tasarlanır.
  - **Birinci Aşama:** Sadece yeni eklenen sınıflandırma katmanları eğitilir ve taban matrisleri korunur.
  - **İkinci Aşama:** İnce ayar uygulanır ve üst katmanlar eğitime açılarak hedef veriye spesifik adaptasyon sağlanır.
- Sonuç olarak ağ genel geçerliliğini korur ve hedef probleme özel çözüm üretir.

Modelin Kaydedilmesi ve Değerlendirilmesi

- Optimizasyon süreci tamamlanan ağ h5 uzantılı formatta çalışma dizinine kaydedilir.
- Doğrulama veri seti üzerinden kayıp ve isabet oranları hesaplanır.

```
model.save('mobilenetv2_transfer.h5')

loss, acc = model.evaluate(val_ds)
print(f"Dogruluk orani: {acc:.2f}")
```

Çıkarım (Inference) Aşaması Nedir?

- Eğitim süreci tamamlanmış bir yapay zeka modelinin önceden görmediği yeni veriler üzerinde tahmin yapma işlemine çıkarım (inference) adı verilir.
- Eğitim aşamasında model ağırlıkları sürekli güncellenirken çıkarım aşamasında ağırlıklar sabittir ve optimizasyon yapılmaz.
- Model sadece ileri yayılım (forward pass) işlemini gerçekleştirir.
- Çıkarım işlemleri eğitim sürecine kıyasla çok daha düşük donanım kaynağı tüketir ve genellikle gerçek zamanlı olarak çalışır.

Eğitilmiş Model ile Tahmin İşlemi

```
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import numpy as np

model = load_model('mobilenetv2_transfer.h5')
img_path = 'example_flower.jpg'

img = image.load_img(img_path, target_size=(160, 160))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

pred = model.predict(img_array)
class_idx = np.argmax(pred)
print("Tahmin edilen sinif indeksi:", class_idx)
```

Karmaşıklık Matrisi (Confusion Matrix) Kurulumu

```
from sklearn.metrics import confusion_matrix,
    classification_report
import seaborn as sns
import numpy as np
val_images = []
val_labels = []
for images, labels in val_ds:
    val_images.append(images.numpy())
    val_labels.append(labels.numpy())
val_images = np.vstack(val_images)
val_labels = np.concatenate(val_labels)
predictions = model.predict(val_images)
predicted_classes = np.argmax(predictions, axis=1)
```

Karmaşıklık Matrisi Görselleştirme

```
cm = confusion_matrix(val_labels, predicted_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Tahmin Edilen Sinif')
plt.ylabel('Gercek Sinif')
plt.title('Karmasiklik Matrisi Analizi')
plt.show()

print(classification_report(val_labels, predicted_classes,
    target_names=class_names))
```

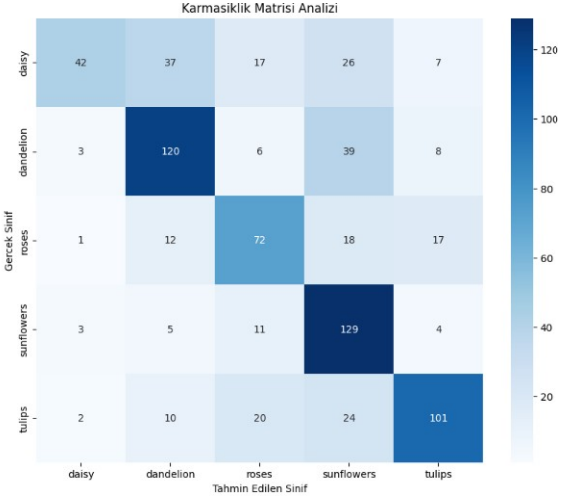
Açıklanabilir Yapay Zeka (Explainable AI)

- Derin öğrenme modelleri genellikle şeffaf olmayan karmaşık matematiksel sistemler olarak çalışır.
- Sınıflandırma kararlarının uzamsal temellerini anlamak için açıklanabilir yapay zeka algoritmaları kullanılır.
- Grad-CAM yöntemi son evrişim katmanındaki gradyan akışını hesaplayarak piksel bazlı ısı haritaları üretir.
- Ağın sınıflandırma işlemi sırasında matrisin hangi bölgesine odaklandığı net olarak tespit edilir.

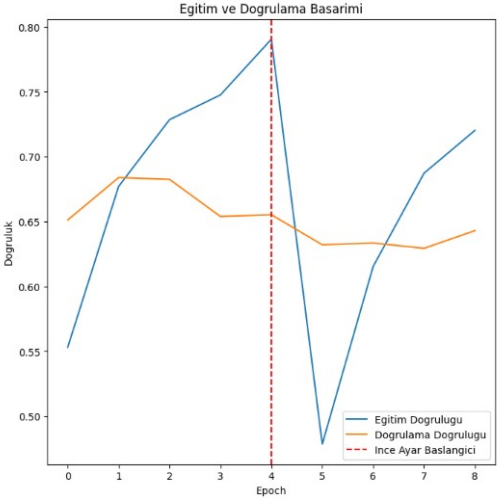
Eğitim Metriklerinin Görselleştirilmesi

```
import matplotlib.pyplot as plt

acc = history_fe.history['accuracy'] + history_ft.history['
    accuracy']
val_acc = history_fe.history['val_accuracy'] + history_ft.
    history['val_accuracy']

plt.figure(figsize=(8, 6))
plt.plot(acc, label='Egitim Dogrulugu')
plt.plot(val_acc, label='Dogrulama Dogrulugu')
plt.axvline(x=4, color='r', linestyle='--', label='Ince Ayar
    Baslangici')
plt.legend(loc='lower right')
plt.title('Egitim ve Dogrulama Basarimi')
plt.xlabel('Epoch')
plt.ylabel('Dogruluk')
plt.show()
```

Görsel


Karmasiklik Matrisi Analizi
Gercek Sinif
daisy
dandelion
roses
sunflowers
tulips
42
37
17
26
7
3
120
6
39
8
1
12
72
18
17
3
5
11
129
4
2
10
20
24
101
daisy
dandelion
roses
sunflowers
tulips
Tahmin Edilen Sinif
120
100
80
60
40
20

Görsel


Egitim ve Dogrulama Basarimi
0.80
0.75
0.70
0.65
0.60
0.55
0.50
Dogruluk
0
1
2
3
4
5
6
7
8
Epoch
Egitim Dogrulugu
Dogrulama Dogrulugu
Ince Ayar Baslangici

Görsel

Grad-CAM Odak Analizi

Renk Kodlamasının Anlamı

- Görsel üzerindeki sıcak renkler yani kırmızı ve sarı tonlar modelin tahmin yaparken en çok kullandığı ve yüksek gradyan değeri üreten pikselleri ifade eder.
- Soğuk renkler olan mavi ve lacivert tonlar ağın sınıflandırma aşamasında matematiksel olarak dikkate almadığı arka plan öğelerini gösterir.

Video İşleme Temel Kavramları

- Video verisi ardışık olarak sıralanmış durağan görüntü karelerinden oluşur.
- Saniyede gösterilen kare sayısı videonun akıcılığını ve donanım üzerinde işlenecek veri miktarını belirler.
- Görüntü işleme algoritmaları videoyu bir bütün olarak değil tekil kareler dizisi olarak ele alır.
- Her bir kare standart matris formatına dönüştürülerek sinir ağına girdi olarak sunulur.

Derin Öğrenme ile Video Analizi

- Video işleme görevleri uzamsal özelliklerin yanında zamansal ilişkilerin de analiz edilmesini gerektirir.
- Evrişimsel sinir ağları kareler üzerindeki nesneleri tespit ederken tekrarlayan sinir ağları zaman içindeki değişimi izler.
- Üç boyutlu evrişimsel ağlar zaman boyutunu matris işlemine doğrudan dahil eder.
- Hareket tespiti ve eylem tanıma görevlerinde bu karmaşık ağ mimarileri kullanılır.

OpenCV ile Video Verisi Okuma

- Video dosyalarını okumak ve anlık kamera verisi almak için OpenCV kütüphanesi kullanılır.

```
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow('Video Ekrani', frame)
    if cv2.waitKey(1) == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

Video Kareleri Üzerinde Model Çıkarımı

```
import cv2
import numpy as np
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    img_resized = cv2.resize(frame, (160, 160))
    img_array = np.expand_dims(img_resized, axis=0)
    pred = model.predict(img_array)
    class_idx = np.argmax(pred)
    cv2.putText(frame, str(class_idx), (50, 50), cv2.
       FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.imshow('Cikarim Ekrani', frame)
    if cv2.waitKey(1) == ord('q'):
        break
cap.release()
```